



COMSATS University Islamabad, Attock Campus
Department of Computer Science

Course: Artificial Intelligence

Project	AI Snake Game
Name	Muhammad Asim Muhammad Ahsan Fawad
Registration Numbers	CIIT/FA24-BAI-030/ATK CIIT/FA24-BAI-038/ATK
Program	BS-AI
Submitted To	Mr. Babar Ali

AI Snake Game

1 Introduction

The **AI Snake Game** is a Python-based game developed using Pygame. This project is inspired by the classic Snake Game but includes an important improvement: an intelligent AI-controlled snake that competes against the human player.

The game is designed to demonstrate how Artificial Intelligence can be used in game development. The AI snake is capable of finding the shortest path toward food, avoiding obstacles, and making safe decisions when trapped.

This project combines:

- Python programming
- Game development
- Artificial Intelligence
- Pathfinding algorithms
- Real-time decision making

2 Project Objective

The main objective of this project is to create an interactive Snake Game where a human player competes against an AI-controlled snake.

The goals of the project are:

- To develop a complete Snake Game using Python
- To implement AI behavior using pathfinding algorithms
- To understand the practical use of the A* Search Algorithm
- To create intelligent movement and collision handling
- To improve programming and problem-solving skills

3 Technologies Used

The following technologies and libraries were used in the project:

Technology Purpose

Python	Main programming language
Pygame	Graphics, game window, and event handling
heapq	Used in A* priority queue
deque	Efficient snake movement
random	Random food generation

4 Game Description

The game contains two snakes:

1. Human-controlled snake
2. AI-controlled snake

The player controls the green snake using keyboard arrow keys, while the blue snake is controlled automatically by AI.

Both snakes try to collect food placed randomly on the grid. Each time a snake eats food, its score increases.

The first snake to reach the winning score wins the game.

The game ends if a snake:

- Hits the wall
- Hits itself
- Collides with the opponent snake

5 Game Features

The project includes several features that make the game interactive and intelligent.

6 Main Features

- Human vs AI gameplay
- Real-time movement
- Intelligent AI pathfinding
- Score tracking system
- Collision detection
- AI path visualization
- Restart and quit options
- Smooth graphical interface

7 Working of the AI

The AI snake is the most important part of this project.

The AI continuously analyzes:

- Food position
- Snake positions
- Obstacles
- Safe movement areas

Based on this information, the AI decides its next move.

The AI uses two main algorithms:

1. A* Search Algorithm
2. Flood Fill Algorithm

8 A* Search Algorithm

The AI first tries to reach the food using the A* Search Algorithm.

A* is one of the most efficient pathfinding algorithms used in games and robotics.

The algorithm calculates the best possible path from the AI snake to the food while avoiding obstacles.

The formula used by A* is:

$$f(n)=g(n)+h(n)$$

Where:

- $g(n)$ represents the actual distance travelled
- $h(n)$ represents the estimated remaining distance

The algorithm selects the path with the lowest total cost.

9 Manhattan Distance Heuristic

The heuristic used in this project is the Manhattan Distance.

It calculates the distance between two grid cells.

Formula:

$$|x_1-x_2|+|y_1-y_2|$$

This heuristic works well in grid-based games because movement is limited to four directions:

- Up
- Down
- Left
- Right

10 Flood Fill Algorithm

Sometimes the AI cannot safely reach the food because obstacles block the path.

In this situation, the AI uses the Flood Fill Algorithm.

The Flood Fill algorithm checks how much free space is available in each direction.

The AI then chooses the safest direction with maximum open space.

This prevents the AI from trapping itself.

11 Game Mechanics

12 Snake Movement

The snakes move cell by cell on a grid.

Each movement updates:

- Snake head position
- Snake body position
- Collision checking

If food is eaten:

- Snake grows longer
- Score increases
- New food is generated

13 Collision Detection

The game continuously checks for collisions.

14 Types of Collisions

15 Wall Collision

Occurs when a snake moves outside the game boundary.

16 Self Collision

Occurs when the snake touches its own body.

17 Opponent Collision

Occurs when one snake collides with the other snake.

18 User Controls

Key	Action
------------	---------------

Arrow Keys	Move snake
------------	------------

Key	Action
R	Restart game
Q	Quit game

19 Graphics and Interface

The graphical interface is created using Pygame.

The interface includes:

- Background grid
- Snake bodies
- Food object
- Scoreboard
- AI path dots
- Game-over screen

Different colors are used to clearly distinguish:

- Player snake
- AI snake
- Food
- Path visualization

20 Data Structures Used

Different data structures are used for efficient gameplay.

Data Structure Purpose

deque	Snake body storage
set	Fast collision checking
list	AI path storage

Data Structure Purpose

heap Priority queue in A*

21 Advantages of the Project

This project provides several learning benefits:

- Demonstrates Artificial Intelligence concepts
- Improves understanding of pathfinding algorithms
- Enhances Python programming skills
- Introduces game development concepts
- Demonstrates real-time problem solving

22 Limitations

Although the project works successfully, it still has some limitations:

- No multiplayer support
- Fixed game environment
- Limited AI strategies
- No sound effects
- No difficulty levels

23 Future Improvements

The project can be improved further by adding:

- Multiple difficulty modes
- Obstacles and maze levels
- Online multiplayer support
- Sound and background music
- Machine Learning-based AI
- Mobile support

- Power-ups and special items

24 Conclusion

The **AI Snake Game** is a successful combination of game development and Artificial Intelligence.

The project demonstrates how algorithms like A* Search and Flood Fill can be used to create intelligent behavior in games.

The AI snake is capable of:

- Finding optimal paths
- Avoiding collisions
- Making survival decisions

This project helped in understanding:

- Python programming
- AI concepts
- Pathfinding techniques
- Real-time game logic

Overall, the project is an effective example of how Artificial Intelligence can be integrated into interactive games.

Code:

```
import pygame
import sys
import random
import heapq
from collections import deque

WINDOW_W, WINDOW_H = 600, 600 # Total window size in pixels
GRID = 20 # Each cell is 20x20 pixels
COLS= WINDOW_W // GRID # 30 columns
ROWS = WINDOW_H // GRID # 30 rows
FPS = 7
AI_MOVE_DELAY = 2
WIN_SCORE = 5
winner = None

# — Colors RGB
BG = (12, 12, 20)
GRID_CLR = (28, 28, 42) # Subtle grid lines
P_HEAD = (80, 230, 90) # Human snake head — bright green
P_BODY = (45, 170, 55) # Human snake body — darker green
AI_HEAD = (70, 140, 240) # AI snake head — bright blue
AI_BODY = (35, 85, 185) # AI snake body — darker blue
FOOD_CLR = (230, 55, 60) # Food — red circle
PATH_CLR = (255, 200, 50) # AI path visualisation — yellow dots
TEXT_CLR = (220, 220, 220) # General UI text
DIM_CLR = (110, 110, 130) # Dimmed secondary text
PANEL_CLR = (20, 20, 35) # Score panel background
WIN_CLR_P = (80, 230, 90) # Win screen colour for Player
WIN_CLR_AI = (70, 140, 240) # Win screen colour for AI
PANEL_H = 50 # Height of the score panel at the top

# — Movement Directions
UP = (0, -1) # Move up: y decreases
DOWN = (0, 1) # Move down: y increases
LEFT = (-1, 0) # Move left: x decreases
RIGHT = (1, 0) # Move right: x increases
DIRS = [UP, DOWN, LEFT, RIGHT] # All possible directions

# Maps each direction to its exact opposite (prevents 180° reversal)
OPPOSITE = {UP: DOWN, DOWN: UP, LEFT: RIGHT, RIGHT: LEFT}

# — A* Search Algorithm
def manhattan(a: tuple, b: tuple) -> int:
    """
    Manhattan Distance Heuristic.
    Estimates cost from cell `a` to cell `b` by summing
    the absolute differences in x and y coordinates.
```

Named after the grid-like streets of Manhattan, New York.

"""

```
return abs(a[0] - b[0]) + abs(a[1] - b[1])
```

```
def astar(start: tuple, goal: tuple, obstacles: set) -> list:
```

```
    open_heap = []          # Min-heap: (f_score, counter, cell)
    counter = 0             # Tie-breaker for equal f-scores
    heapq.heappush(open_heap, (manhattan(start, goal), counter, start))
```

```
    came_from = {}         # Tracks how we reached each cell
    g_score = {start: 0}   # Actual cost from start to each visited cell
```

```
    while open_heap:
```

```
        _, _, cur = heapq.heappop(open_heap) # Pop cell with lowest f-score
```

```
        if cur == goal:
```

```
            # Goal reached — reconstruct the path by backtracking
```

```
            path = []
```

```
            while cur in came_from:
```

```
                path.append(cur)
```

```
                cur = came_from[cur]
```

```
            path.reverse()
```

```
            return path
```

```
    # Explore all 4 neighbours (up, down, left, right)
```

```
    for dx, dy in DIRS:
```

```
        nb = (cur[0] + dx, cur[1] + dy)
```

```
        # Skip if out of bounds
```

```
        if not (0 <= nb[0] < COLS and 0 <= nb[1] < ROWS):
```

```
            continue
```

```
        # Skip if blocked
```

```
        if nb in obstacles:
```

```
            continue
```

```
        tg = g_score[cur] + 1 # All moves cost 1
```

```
        if tg < g_score.get(nb, float('inf')):
```

```
            came_from[nb] = cur
```

```
            g_score[nb] = tg
```

```
            f = tg + manhattan(nb, goal)
```

```
            counter += 1
```

```
            heapq.heappush(open_heap, (f, counter, nb))
```

```
    return [] # No path found
```

```
# — Flood Fill (Safe Fallback) —————
```

```
def flood_fill(start: tuple, obstacles: set) -> int:
```

```
    """
```

BFS-based flood fill — counts all reachable cells from `start`.

The AI uses this as a survival heuristic:

when it can't reach the food, it moves toward
whichever direction has the most open space.

Returns: integer count of reachable cells

```
"""
visited = {start}
queue = deque([start])
while queue:
    cx, cy = queue.popleft()
    for dx, dy in DIRS:
        nb = (cx + dx, cy + dy)
        if (0 <= nb[0] < COLS and 0 <= nb[1] < ROWS
            and nb not in obstacles and nb not in visited):
            visited.add(nb)
            queue.append(nb)
return len(visited)
```

— AI Decision Engine —————

```
def ai_next_cell(ai_snake: deque, human_snake: deque, food: tuple):
    """
```

Decides the AI snake's next move each game tick.

Strategy (in priority order):

1. Run A* toward the food.
 - Obstacle set EXCLUDES tails of both snakes,
because tails will move away by the time AI arrives.
2. If A* returns no path, use flood-fill fallback:
 - Look at all 4 neighbours of AI head.
 - Pick the neighbour with the most reachable open space.
 - This keeps the AI alive as long as possible.

Returns:

(next_cell, path_list)

next_cell is None if the AI is completely trapped.

```
"""
```

```
head = ai_snake[0]
```

Build obstacle sets, excluding tails (they vacate next step)

```
ai_obs = set(ai_snake) - {ai_snake[-1]}
```

```
human_obs = set(human_snake) - {human_snake[-1]}
```

```
obstacles = ai_obs | human_obs
```

— Primary strategy: A* to food —————

```
path = astar(head, food, obstacles)
```

```
if path:
```

```
    return path[0], path # First cell in path = next move
```

```
# — Falback: maximise open space —————
best, best_space = None, -1
full_obs = set(ai_snake) | set(human_snake)
for dx, dy in DIRS:
    nb = (head[0] + dx, head[1] + dy)
    if (0 <= nb[0] < COLS and 0 <= nb[1] < ROWS
        and nb not in full_obs):
        space = flood_fill(nb, full_obs)
        if space > best_space:
            best_space, best = space, nb
return best, [] # None if completely trapped

# — Game State Helpers —————
def spawn_food(snake1: deque, snake2: deque) -> tuple:
    """
    Randomly places food on an unoccupied grid cell.
    Keeps trying until it finds a cell not occupied by either snake.
    """
    occupied = set(snake1) | set(snake2)
    while True:
        pos = (random.randint(0, COLS - 1), random.randint(0, ROWS - 1))
        if pos not in occupied:
            return pos

def init_game():
    """
    Initialises / resets all game state to starting positions.

    - Human snake starts on the LEFT side, heading RIGHT
    - AI snake starts on the RIGHT side, heading LEFT
    - Food is placed randomly on an empty cell
    - Both scores reset to 0

    Returns a tuple of all game state variables.
    """
    # Human: 3-cell snake at left-centre
    p_start = (COLS // 4, ROWS // 2)
    p_snake = deque([p_start,
                     (p_start[0] - 1, p_start[1]),
                     (p_start[0] - 2, p_start[1])])
    p_dir = RIGHT

    # AI: 3-cell snake at right-centre
    ai_start = (3 * COLS // 4, ROWS // 2)
    ai_snake = deque([ai_start,
                      (ai_start[0] + 1, ai_start[1]),
                      (ai_start[0] + 2, ai_start[1])])
```

```
food = spawn_food(p_snake, ai_snake)
p_score = ai_score = 0
return p_snake, p_dir, ai_snake, food, p_score, ai_score
```

— Drawing Functions —————

```
def grid_rect(x: int, y: int) -> pygame.Rect:
    """Converts grid (col, row) coordinates to a pixel Rect for drawing."""
    return pygame.Rect(x * GRID, PANEL_H + y * GRID, GRID, GRID)
```

```
def draw_bg(surf: pygame.Surface) -> None:
    """Fills background and draws faint grid lines on the game area."""
    surf.fill(BG)
    for x in range(0, WINDOW_W, GRID):
        pygame.draw.line(surf, GRID_CLR, (x, PANEL_H), (x, WINDOW_H))
    for y in range(PANEL_H, WINDOW_H, GRID):
        pygame.draw.line(surf, GRID_CLR, (0, y), (WINDOW_W, y))
```

```
def draw_snake(surf: pygame.Surface, snake: deque,
               head_clr: tuple, body_clr: tuple) -> None:
    """
    Draws a snake on the surface.
    Head (index 0) uses a brighter colour; body uses a darker shade.
    Each cell is drawn as a rounded rectangle.
    """
    for i, (x, y) in enumerate(snake):
        clr = head_clr if i == 0 else body_clr
        rect = grid_rect(x, y).inflate(-2, -2) # Slight inset for spacing
        pygame.draw.rect(surf, clr, rect, border_radius=5)
```

```
def draw_food(surf: pygame.Surface, food: tuple) -> None:
    """Draws the food as a red circle centred in its grid cell."""
    r = grid_rect(*food)
    pygame.draw.circle(surf, FOOD_CLR, r.center, GRID // 2 - 2)
```

```
def draw_path(surf: pygame.Surface, path: list) -> None:
    """
    Visualises the AI's planned A* path as small yellow dots.
    This helps the human player see where the AI intends to go.
    """
    for (px, py) in path:
        r = grid_rect(px, py)
        pygame.draw.circle(surf, PATH_CLR, r.center, GRID // 6)
```

```
def draw_panel(surf: pygame.Surface, font: pygame.font.Font,
               p_score: int, ai_score: int) -> None:
    """
    Draws the top score panel showing:
    - Player score (green)
    - AI score (blue)
    """
```

```

- Target score (dimmed)
"""

pygame.draw.rect(surf, PANEL_CLR, (0, 0, WINDOW_W, PANEL_H))
pygame.draw.line(surf, GRID_CLR, (0, PANEL_H), (WINDOW_W, PANEL_H), 2)

p_lbl = font.render(f"Player: {p_score}", True, P_HEAD)
ai_lbl = font.render(f"AI: {ai_score}", True, AI_HEAD)
win_lbl = font.render(f"First to {WIN_SCORE}", True, DIM_CLR)

surf.blit(p_lbl, (16, (PANEL_H - p_lbl.get_height()) // 2))
surf.blit(ai_lbl, (220, (PANEL_H - ai_lbl.get_height()) // 2))
surf.blit(win_lbl, (400, (PANEL_H - win_lbl.get_height()) // 2))

def draw_end_screen(surf: pygame.Surface,
                    big: pygame.font.Font, sm: pygame.font.Font,
                    winner: str, p_score: int, ai_score: int) -> None:
    """
    Draws a semi-transparent overlay when the game ends, showing:
    - Winner name (coloured to match their snake)
    - Final scores
    - Restart / Quit instructions
    """
    overlay = pygame.Surface((WINDOW_W, WINDOW_H), pygame.SRCALPHA)
    overlay.fill((0, 0, 0, 170)) # Black with ~67% opacity
    surf.blit(overlay, (0, 0))

    clr = WIN_CLR_P if winner == "Player" else WIN_CLR_AI
    txt1 = big.render(f"{winner} Wins!", True, clr)
    txt2 = sm.render(f"Player {p_score} - AI {ai_score}", True, TEXT_CLR)
    txt3 = sm.render("R - Restart   Q - Quit", True, DIM_CLR)

    cx, cy = WINDOW_W // 2, WINDOW_H // 2
    surf.blit(txt1, txt1.get_rect(center=(cx, cy - 55)))
    surf.blit(txt2, txt2.get_rect(center=(cx, cy + 5)))
    surf.blit(txt3, txt3.get_rect(center=(cx, cy + 55)))

# — Main Game Loop —
def main():
    """
    Main entry point. Initialises Pygame and runs the game loop.
    """
    pygame.init()
    screen = pygame.display.set_mode((WINDOW_W, WINDOW_H))
    pygame.display.set_caption("Snake Battle - Human vs AI")
    clock = pygame.time.Clock()

    # Load fonts (Consolas is monospaced — great for game UIs)
    big_font = pygame.font.SysFont("consolas", 52, bold=True)
    med_font = pygame.font.SysFont("consolas", 26)

```

```

sm_font = pygame.font.SysFont("consolas", 20)

# Initialise game state
(p_snake, p_dir,
 ai_snake, food,
 p_score, ai_score) = init_game()

pending_p_dir = p_dir # Buffer: stores the next direction before tick fires
ai_path = [] # A* path to visualise as yellow dots

winner = None # None while game is running
ai_move_counter = 0

# -----
# GAME LOOP
# -----
while True:

    # --- Event Handling -----
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            pygame.quit()
            sys.exit()

        if event.type == pygame.KEYDOWN:
            if event.key == pygame.K_q:
                pygame.quit()
                sys.exit()

            # R key restarts the game (only works after game over)
            if event.key == pygame.K_r and winner:
                (p_snake, p_dir,
                 ai_snake, food,
                 p_score, ai_score) = init_game()
                pending_p_dir = p_dir
                ai_path = []
                winner = None
                continue

            # Arrow keys control the human snake
            # OPPOSITE check prevents the snake reversing into itself
            if not winner:
                key_map = {
                    pygame.K_UP: UP,
                    pygame.K_DOWN: DOWN,
                    pygame.K_LEFT: LEFT,
                    pygame.K_RIGHT: RIGHT,
                }
                new_dir = key_map.get(event.key)

```

```

        if new_dir and new_dir != OPPOSITE.get(p_dir):
            pending_p_dir = new_dir # Buffer the input

# — Game-Over Screen (no movement, just redraw) —————
if winner:
    draw_bg(screen)
    draw_path(screen, ai_path)
    draw_food(screen, food)
    draw_snake(screen, p_snake, P_HEAD, P_BODY)
    draw_snake(screen, ai_snake, AI_HEAD, AI_BODY)
    draw_panel(screen, sm_font, p_score, ai_score)
    draw_end_screen(screen, big_font, med_font, winner, p_score, ai_score)
    pygame.display.flip()
    clock.tick(FPS)
    continue

# — AI Decision (computed BEFORE movement) —————
# The AI sees the current state and picks its next cell
ai_next, ai_path = ai_next_cell(ai_snake, p_snake, food)

# — Move Human Snake —————
p_dir = pending_p_dir # Apply buffered direction
p_head = (p_snake[0][0] + p_dir[0], # Calculate new head position
          p_snake[0][1] + p_dir[1])
p_snake.appendleft(p_head) # Add new head to front
p_ate = (p_head == food) # Did player eat food?
if not p_ate:
    p_snake.pop() # Remove tail if no food eaten

# — Move AI Snake —————
# — Move AI Snake (Fair Speed) —————
ai_ate = False
ai_move_counter += 1
# AI moves slower for fair gameplay
if ai_move_counter >= AI_MOVE_DELAY:
    ai_move_counter = 0
    if ai_next:
        ai_snake.appendleft(ai_next)
        ai_ate = (ai_next == food)
    if not ai_ate:
        ai_snake.pop()

# — Collision Detection —————
def is_dead(snake: deque, other: deque) -> bool:
    """Returns True if the snake's head has collided with something."""
    head = snake[0]
    # 1. Hit a wall
    if not (0 <= head[0] < COLS and 0 <= head[1] < ROWS):
        return True

```

```

# 2. Hit own body (skip index 0 which is the head itself)
if head in list(snake)[1:]:
    return True
# 3. Hit the other snake
if head in set(other):
    return True
return False

p_dead = is_dead(p_snake, ai_snake)
ai_dead = is_dead(ai_snake, p_snake) or (ai_next is None)

# Determine winner from collision results
if p_dead and ai_dead:
    winner = "Nobody – it's a draw!"
elif p_dead:
    winner = "AI"
elif ai_dead:
    winner = "Player"

# — Score Updates —————
if not winner:
    if p_ate:
        p_score += 1
        food = spawn_food(p_snake, ai_snake) # Spawn new food
        ai_path = [] # Clear old AI path
    elif ai_ate:
        ai_score += 1
        food = spawn_food(p_snake, ai_snake)
        ai_path = []

    if p_score >= WIN_SCORE:
        winner = "Player"
    elif ai_score >= WIN_SCORE:
        winner = "AI"

# — Render Frame —————
draw_bg(screen)
if ai_path:
    draw_path(screen, ai_path[1:]) # Skip dot on immediate next cell
draw_food(screen, food)
draw_snake(screen, p_snake, P_HEAD, P_BODY)
draw_snake(screen, ai_snake, AI_HEAD, AI_BODY)
draw_panel(screen, sm_font, p_score, ai_score)

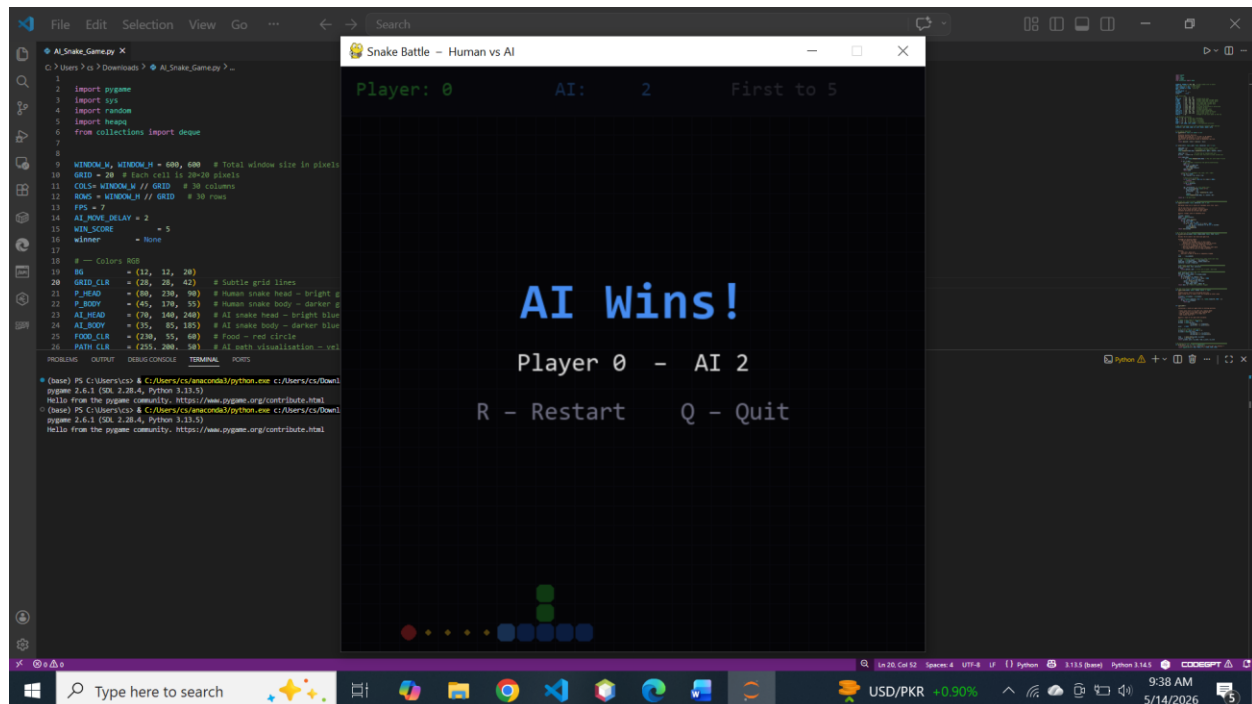
pygame.display.flip() # Push the frame to the screen
clock.tick(FPS) # Enforce 6 FPS tick rate

# — Entry Point —————
if __name__ == "__main__":

```

```
main()
```

Output:



Player: 0

AI: 0

First to 5

